

Distributed event queue management system

Patryk Nowacki

Warsaw, Poland

p.g.nowacki@protonmail.com

Radosław Roszczyk  Member, IEEE,

Warsaw University of Technology

Warsaw, Poland

Artur Krupa 

Warsaw University of Life Sciences

Warsaw, Poland

Abstract—This document describes innovative ways of storing events in queues and the working principle of the event queue management system. The paper also includes a description of methods preventing accidents of the event queue management system. The work also presents example scenarios of using the proposed system.

Index Terms—distributed system, events, queue system

I. INTRODUCTION

Management of continuously growing data in digital systems has a severe matter of effectiveness and its functionality. Network communication requires fast and dynamic data transfer regardless of media issues, possible delay, or trace routing of packages. Waiting over a defined time for incoming information is unacceptable and requires transmission repetition. A great example of a nearly perfectly implemented data split solution among multiple nodes and optimized transfer is TCP/IP Protocol. It maximizes the capabilities of the transmission system and minimizes current issues nearly to zero. Use of buffering data packets, implementation of the definition of packet existence (Time-To-Live parameter), redirection through the network to find the best route are precisely implemented solutions that pass the exam.

The problem occurs when it is necessary to accomplish data transmission away from the direct network protocol - in monolith-based systems, or integrated circuits/systems. This is why Queue Management System is desirable in this situation. Algorithms that describe well-working solutions are not widely used. Most of them are already described as patented or waiting to be patented in public documentation and can not be cited. Among these definitions, there is a generally available mechanism that all of them use - called AQM (Active Queue Management) [1], that is based on RED (Random Early Detection) scheme [2] with its modifications taking account of the sensitivity parameter.

Interesting is also defined an AQM implementation with Linear Quadratic Regulator (LQR) method in [3] combined with PID controller that realizes "responsiveness and minimum steady-state error means stable queue length." Finally, the implementation of the idea of Queue Management was described in [4], which was proposed as a real-time service based on IoT devices (Arduino platform supported with WebRTC technology).

This article presents an innovative model of a Queue Management System that allows for effective and reliable

managing resources connected to a queue system. It contains an introduction to the fundamentals of the system, information that describes each of the types of queuing, and a description of typical scenarios associated with the designed Queue Management System. The article was summarized with the advantages and disadvantages of the system.

II. MANAGEMENT SYSTEM

The Queue Management Service proposed here is connected with managing data stored in queues. Data defined here are called "messages." The idea is based on RabbitMQ idea. Each message is processed by Queue Manager, which defines the importance of each message and creates a properly defined queue. Each queue contains zero or more messages, and messages can be stored or taken from the queue. The message itself contains binary data that can be decoded for further processing in a queuing system. Message processing can be considered finished if the sender has processed no further existing relation or last delivery confirmation. For more efficient processing, each message in the system has its own identifier (ID).

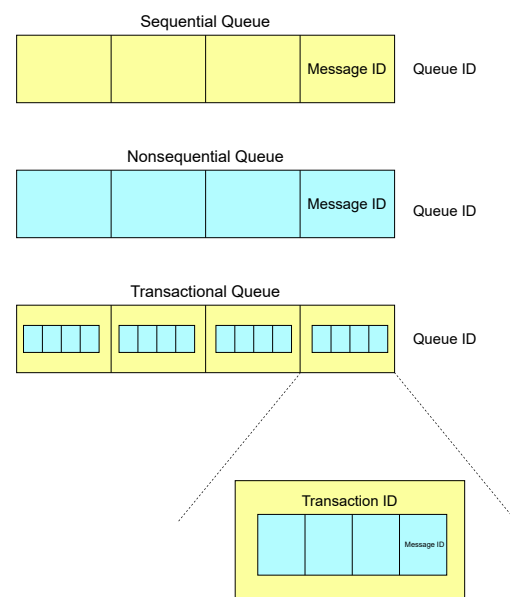


Fig. 1. Three types of Queues structure

Each message has three possible states: ready, processing, removed. The "ready" state defines the message as "put into a system and wait for the system queue". The "processing"

state defines the message as "taken from the queue" but not admitted yet, so the process does not finish the task now. The "removed" state declares that the message has been processed and confirmed.

III. TYPES OF QUEUING

There are widely known three types of queuing systems - nonsequential, sequential, and transactional. Each of these systems was designed to meet the requirements of queue processing models.

To explain the distributed event management system, we have to show how each of the queuing is operating.

The first type of queuing is Nonsequential Queue, which is connected with no sequential actions - the next message is taken from the queue regardless of success or failure in processing the previous one. Each message ready to be processed is taken from the queue and consumed consecutively without acknowledging the status. Based on the presented scheme Fig.2 two messages have been processed and probably finished with success.

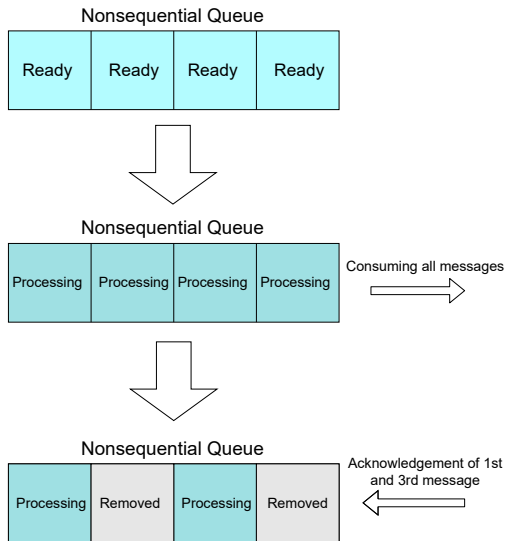


Fig. 2. Nonsequential Queue structure

The second type of queuing is Sequential Queues connected directly with sequential actions, which forces the system to take the next message from the queue only on the previous success state. As presented in Fig.3 all messages at the beginning are in a ready state after this queue system takes the first message and consumes it to a processing state. After the message status returns to the queue (or not, after the declared time), the next message can be consumed from the queue to a processing state.

The last queuing system is Transactional Queue, a hybrid of the Nonsequential Queue and the Sequential Queues. All messages sent to a queuing system are divided into more significant messages containing submessages only related to their parent. More significant messages are consumed in a processing state in Sequential Queues. The next block is

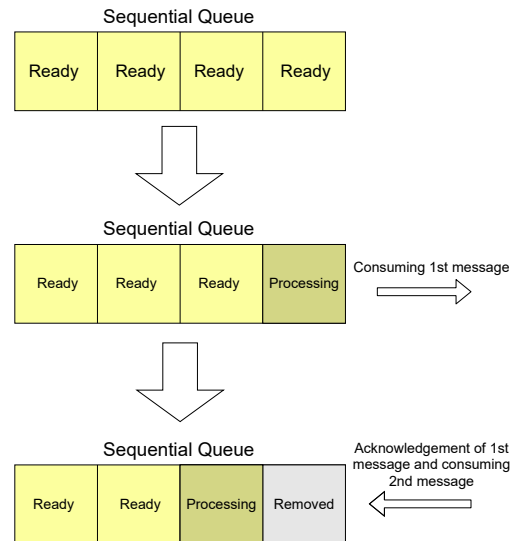


Fig. 3. Sequential Queue structure

consumed after taking information about the previous one processed with success or failure. Each larger message contains a set of submessages consumed in one transaction without the necessity of waiting for a single message. Success in the processing of a transaction were set after all messages in transaction were processed. The processing diagram has been presented in Fig.4.

IV. DISTRIBUTED ARCHITECTURE

The designed queue management system is a type of distributed architecture and consists of sectors and modules. A sector is a group of modules that provide the same functionality. Distribution is that modules interact with each other by using network protocols, and each sector can be developed regardless of the other sectors, but according to its functionality.

Designed and presented in this article, the queue management system consists of the following modules:

- **Producer** - initialized queues in the queue service puts messages into queues for further jobs. After that, Queue Manager sends the queue ID to an instance of Job Manager. Job Manager starts to take messages from a queue with the given ID, despite this queue type.
- **Job Manager** - takes messages from the queue specified by the queue ID and sends them to an instance of the worker. Job Manager also processes feedback from Ack Manager whether the message is processed correctly and returns a success or failure message. It manages the message state to "removed" or "processing again" in case of an issue.
- **Worker** - processing message provided by Job Manager. After successful processing, the worker has to confirm success or failure by informing Ack Manager.
- **Ack Manager** - waiting to the end of processing the specified message by any instance of worker. After the

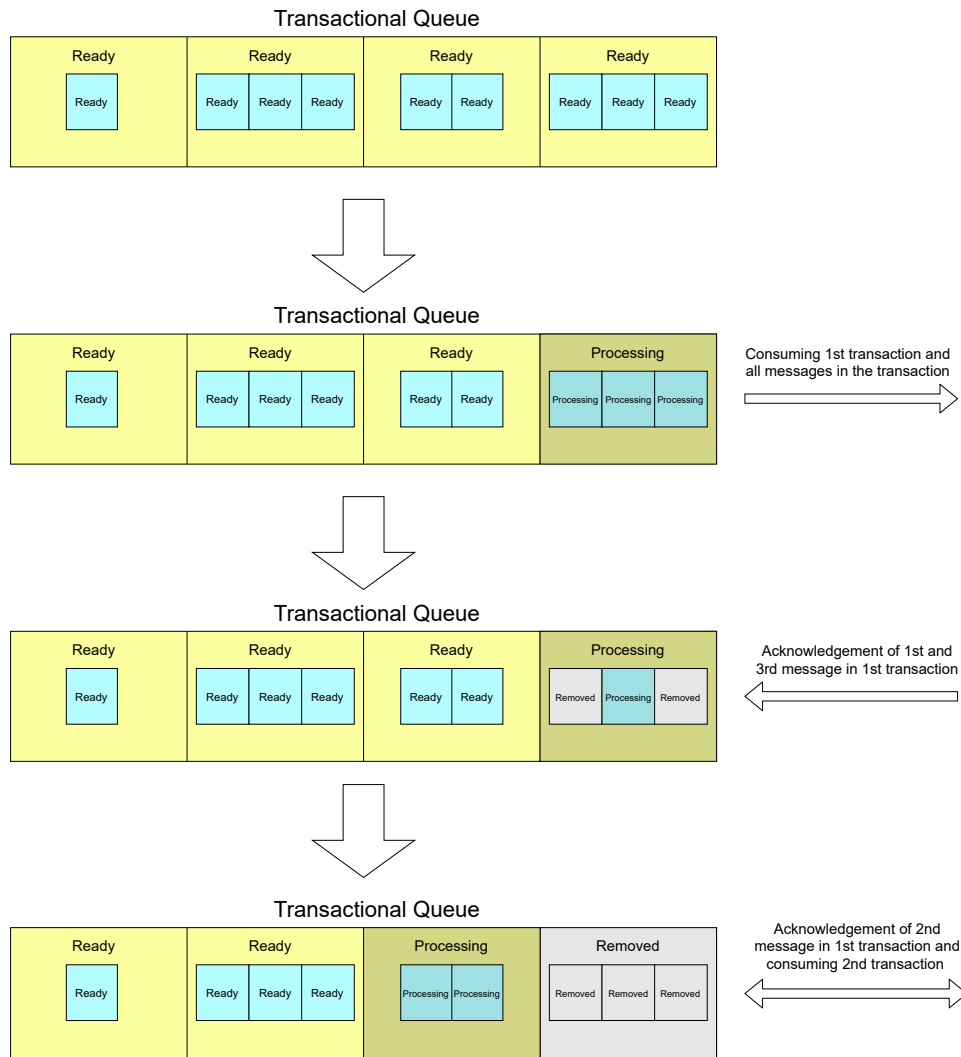


Fig. 4. Transactional Queue structure

end, the worker informs the Ack Manager about the success or failure of the message. Next, Ack Manager is sending information to the Job Manager about the end of the message processing.

- Database - contain crucial information about the message management process and modules responsible for supervision and job execution. These are:
 - Job Manager ID - indicates which instance of Job Manager received a specified message from a queue.
 - Worker ID - indicates which instance of the worker is processing a specified message.
 - Queue ID - indicates to which queue in Queue Service the processed message is related.
 - Transaction ID - indicates to which transaction the processed message is related. If the queue with Queue ID is not the Transactional Queue, then Transaction ID receives a special value.
 - Message ID - indicates which message is processed in the queue.

V. DESCRIPTION OF FUNCTIONALITY

To better understand the processing system, the system and the steps must be described in the process. At first, the producer puts messages into the queue with QueueID. Next, Queue Manager sends Queue ID to any instance of Job Manager. After that, the Job Manager starts consuming the queue with the given Queue ID. The consumption process depends on the queue type and is connected to making a copy of a message for further processing. Then, the Job Manager starts a new instance of a worker, updates the database with a new record associated with the processing message, and sends the message to the worker and the unique ID (Job Manager ID). Subsequently, Worker registers itself in AckManager, processes the message, and after the end of that process, Worker informs Ack Manager about it and passes the Job Manager ID to Worker ID and it. Hence, Ack Manager informs Job Manager specified by Job Manager ID about the acknowledging the message processed by Worker, specified by Worker ID. Finally, the Job Manager deletes the

record associated with the processed message and removes the message from Queue Service.

There is a probability that an accident can occur for some module, i.e., lost connection with the system or a technical problem. The designed queue management system is resistant to those situations. For the introduced management queue system, some following methods prevent undesirable situations:

- Security strategy of Job Manager - related to two situations. The first is taking a copy of a message from a queue, passing it to the worker. The second is receiving information from an instance of Ack Manager and removing a message from the queue.

To prevent the effects of the first case, the Job Manager creates a checkpoint after receiving the Queue ID from Queue Manager. The checkpoint lasts until Job Managers pass a message along with an instance of a worker. When the accident happens in that period, the emergency part of the Job Manager will make a rollback discarding all changes committed after the checkpoint. After that, the new instance of Job Manager starts and gains control of the controller's queue by the lost Job Manager.

To avoid the second situation, we do repetitive signals from the Ack Manager instance and create a checkpoint. The checkpoint is created after the signal of the Ack Manager. It lasts until the Job Manager removes a message from the queue. When an instance of Job Manager loses connection with the system, the emergency part of the Job Manager will make a rollback discarding all changes committed after the checkpoint. Then, Ack Manager is informing the other instance of Job Manager about acknowledging the message as long as the instance of Job Manager gives Ack Manager a return callback to stop it.

- Security Strategy of Worker - is simple and relies on creating a new worker instance. The instance of the worker takes over processing a message that the lost worker processed before losing his connection to the system. When the accident happens, Ack Manager handles the connection error associated with the worker and sends information to Job Manager about the accident. Then Job Manager checks the database to determine which message was processed before the accident. Later that, the message is recopied from the queue. After that, the processing message is targeted at the new instance of the worker.
- Security Strategy of Ack Manager - relies on repetitive signals from the instance of workers. If an accident of Ack Manager happens, Worker informs the other instance of Ack Manager about the end of the processing message. If Ack Manager sends the message to Job Manager successfully, Ack Manager sends a return callback to Worker to stop signalling.

VI. EXAMPLE - BASIC SCENARIO

Producer initializes two queues: Sequential Queue (q1) and Nonsequential Queue (q2). The producer puts two messages (m1 and m2) into q1 and two other messages (m3 and m4)

into q2. There are two instances of Job Manager (jm1 and jm2). Queue Manager gives the ID of q1 to jm1 and the ID of q2 to jm2. Next, some activities happen simultaneously:

- jm1 takes m1 from q1, changes the state of m1 from the ready state to the processing state, and gives m1 to an instance of Worker (w1). Jm1 also stores the IDs of jm1, q1, m1, and w1. After the end of processing m1, w1 informs an Ack Manager (a1) instance about it. Hence, a1 informs jm1 about the acknowledgement of processing m1. Then, jm1 removes data from the database associated with IDs of the process and changes the state of m1 from processing to removal. In the next step, jm1 does take m2 from q1. The processing of m2 will be analogous to the processing of m1 with the existence of the Worker (w2).
- jm2 takes m3 from q2, changes the state of m3 from ready to processing, and gives m3 to an instance of Worker (w3). Jm2 also stores the IDs of jm2, q2, m3, and w3. Next, whatever would happen to m3, jm2 takes m4 from q2, changes the state of m4 from ready to processing, and gives m4 to an instance of Worker (w4). Hence, jm2 stores the IDs of jm2, q2, m4, and w4. When any of the worker instances processes m3 or m4, w3 or w4 informs about Ack Manager's instance (a1). The a1 informs jm2 about the acknowledgement of processing m3 or m4. Then, jm2 removes data from the database associated with IDs of the process and changes the state of m3 or m4 from processing to removal.

VII. CONCLUSIONS

Commonly known architectures of distributed event queue management systems are different than presented here. This has some important advantages:

- provides full control and verification of processing messages from queues,
- provides quite failure-free work.

The designed queue management system contains mechanisms that allow for full control of message processing execution. Entering Ack Management Sector into the queue management system provides verification and proper reaction to message processing.

Modules of the designed queue management system define signalling mechanisms to prevent unpredictable events associated with connection loss or hardware failures. Each receiving module has to confirm its actions by giving a return callback to the sending module.

REFERENCES

- [1] S. S. Kunniyur and R. Srikant, "An Adaptive Virtual Queue (AVQ) algorithm for Active Queue Management," *IEEE/ACM Transactions on Networking*, vol. 12, no. 2, pp. 286–299, 2004.
- [2] V. Firoiu and M. Borden, "Study of active queue management for congestion control," *Proceedings - IEEE INFOCOM*, vol. 3, no. c, pp. 1435–1445, 2000.
- [3] D. Agrawal and F. Granelli, "Redesigning an active queue management system," *GLOBECOM - IEEE Global Telecommunications Conference*, vol. 2, pp. 702–706, 2004.
- [4] M. Ghazal, R. Hamouda, and S. Ali, "An IoT smart queue management system with real-time queue tracking," *Proceedings - 2015 5th International Conference on e-Learning, ECONF 2015*, pp. 257–262, 2016.